ROUND 11 — CONSTITUTION V3 FEATURE SPECIFICATION

The Living Intelligence Layer

Real features for the coder to build — not philosophy, engineering.

Reviewer	Independent (Super Z, Z.ai) — acting as Principal Engineer specifying features under Constitution V3
Commit baseline	df09b49 — docs(constitution-v3): rewrite with 10 Laws + cognitive organs + acceptance criteria
Current score	8.5/10 Constitution adherence (round 10). Security: YES. Test suite: 389 pass, CI-verified.
V3 shift	V2 was frontend collapse. V3 demands backend cognitive organs (Perception, Memory, Understanding, Reasoning, Judgment, Preparation, Reflection, Evolution). 49 backend modules exist; 0 are organized as organs.
Deliverable	7 features. Each has: the V3 law, the gap, exact files, API contract, acceptance test, score delta, effort. The coder builds these. The auditor verifies.

HOW TO READ THIS DOCUMENT

Constitution V3 says "Every line of code must make the organization more intelligent." This is not a platitude — it is a build instruction. The 7 features below are the concrete engineering work that turns V3 from a document into a product. Each feature is grounded in the actual codebase: I read the 49 backend modules, the 19 frontend files, and the API routes before specifying them. Nothing here is vapour.

Order matters. Features #1-#3 are foundational (they create the cognitive organs V3 demands). Features #4-#5 are the user-facing narratives (they replace dashboards with stories). Feature #6 is the time-axis (V3's "make time visible"). Feature #7 is the evolution layer (V3's "organization becomes progressively smarter"). Build them in order — each builds on the previous.

Every feature has an acceptance test I will run. No feature is "done" because the coder says so. It is done when the test passes. The tests are specific: API endpoints, response shapes, grep verifications, live smoke calls. The coder should run the acceptance test before claiming the feature is delivered.

The recurring pattern to break: across 10 rounds, the coder has built tools and not applied them (humanize utility built, not called by deep surfaces; escapeJs built, not applied to 3 onclick handlers; tenant guard built, no-op in single-tenant mode). V3 features MUST be applied, not just built. Each acceptance test checks application, not existence.

Feature Summary

#	Feature	V3 Law	Effort	Score Delta	Dependency
1	Organ So What? Engine	Law 8 (everything answers "so what?")	1 day	+0.5	None
2	Organizational Personality	Law 6 (orgs evolve; infer personality)	2 days	+1.0	None
3	Time-Axis Insight Layer	Law: make time visible	1.5 days	+0.5	Feature #1
4	Conversational Ask (LLM-backed)	Law: replace search with conversation	2 days	+1.0	Feature #1
5	Narrative Dashboard Replacer	Law 1 (interface shrinks; stories replace charts)	1 day	+0.5	Feature #1
6	Deep-Surface Humanize (close R10 gap)	Law 7 (never show machinery)	0.5 day	+0.5	None
7	Quarterly Evolution Report	Law 10 (org becomes progressively smarter)	1.5 days	+1.0	Feature #2
TOTAL	**7 features**	**6 of 10 V3 Laws**	**~9.5 days**	**+5.0 (→ 10/10 cap 9.5)**	

Build order: #6 (half-day, closes R10 gap) → #1 (foundational, everything depends on it) → #2 (personality) → #3 (time-axis) → #4 (conversational ask) → #5 (narrative replacer) → #7 (evolution report, depends on #2). Total ~9.5 days. If all 7 are delivered and verified, Constitution adherence reaches 9.5/10. The 0.5 to 10/10 requires ambient integration (Chrome extension) — post-pilot.

Feature #1 — The "So What?" Engine (Foundational)

FEATURE #1 — **SoWhatEngine: every insight answers "so what?" with consequence + action + timeline**

V3 Law embodied

Law 8: "Everything must answer 'So what?' No metric exists without meaning. No graph exists without consequence. No insight exists without action." Law 9: "Every feature must reduce thinking while increasing judgment."

Current codebase gap

The current Recommendation model (decision.py) has an `impact` field, but it is a static string ("Resolving this gate could unblock 3 in-flight items"). There is no engine that takes ANY insight (recommendation, law, contradiction, risk, prediction) and synthesizes: (a) what happens if ignored, (b) what action to take, (c) when it matters. The CEO briefing shows 8 "money drains" but each is just a title + estimated_cost. There is no "so what" layer. V3 demands every insight answer "so what?" automatically.

Files to create/modify

CREATE `backend/maestro_oem/sowhat.py` — the SoWhatEngine class. MODIFY `backend/maestro_api/routes/oem.py` — add GET /api/oem/sowhat endpoint that accepts an entity_type + entity_id and returns the synthesized "so what" for any insight. MODIFY `backend/maestro_oem/decision.py` — wire SoWhatEngine into Recommendation generation so every rec has a `so_what` field. MODIFY `static/js/today.js` — display the so_what as the brief item's context (replacing the static provenance). MODIFY `static/js/drill_down_modal.js` — add a "So what?" tab that calls the endpoint.

API contract

GET /api/oem/sowhat?entity_type=recommendation&entity;_id=rec-dae18558 → returns JSON: { "consequence_if_ignored": "Velocity will drop 22% next week (3 P1 incidents active).", "recommended_action": "Pause non-critical work and address the payments-edge incident cluster.", "time_horizon": "within 7 days", "confidence_in_consequence": 0.78, "evidence_count": 12, "linked_laws": ["L-0001"] }. The engine synthesizes from: the recommendation's linked laws, the model's health metrics, the evidence graph, and the prediction market (if a prediction exists for this domain). Must work for entity_type in {recommendation, law, contradiction, risk, prediction}.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/sowhat?entity_type=recommendation&entity;_id=. Response must have all 6 fields non-empty. 2) Auditor calls with entity_type=law&entity;_id=L-0001. Response must reference the law's outcome and consequence. 3) Auditor greps decision.py for so_what field on Recommendation — must be populated. 4) Auditor opens TODAY surface and verifies at least one brief item shows a "so what" consequence (not just a title). 5) Auditor opens drill-down modal on any recommendation and verifies a "So what?" tab exists with content.

Score delta

+0.5 (8.5 → 9.0). This is the foundational V3 feature — every other narrative feature depends on it. Without "so what," narratives are just renamed dashboards.

Estimated effort

1 day (0.5 day engine + 0.5 day API + frontend wiring)

Feature #2 — Organizational Personality (Inferred, Never Surveyed)

FEATURE #2 OrganizationalPersonality: infer 6 personality dimensions from behavior, never ask the user

V3 Law embodied

Law 6: "Organizations evolve. Maestro must evolve with them." V3: "After sufficient history, Maestro should understand the organization's personality... Never ask users to fill out surveys. Infer it from behavior."

Current codebase gap

The codebase has no personality module. The `instrumentation.py` file mentions "Principles, Genome, Gravity, Fragility" as future aspirations but implements none. The model has signals, learning objects, laws, contradictions — enough behavioral data to infer personality, but no inference engine. V3 demands the organization feel "understood," which requires Maestro to know HOW the organization makes decisions, not just WHAT it decided.

Files to create/modify

CREATE `backend/maestro_oem/personality.py` — the OrganizationalPersonality class. Infer 6 dimensions, each 0.0-1.0 with a human label: (1) decision_velocity (fast/slow), (2) risk_appetite (cautious/bold), (3) knowledge_mobility (siloed/fluid), (4) meeting_dependency (autonomous/collaborative), (5) review_discipline (loose/rigorous), (6) learning_velocity (stagnant/accelerating). Each dimension computed from existing model data: decision_velocity from approval bottleneck timing, risk_appetite from P1 incident frequency vs release frequency, knowledge_mobility from cross-domain signal overlap, meeting_dependency from gmail calendar signals, review_discipline from PR review rates, learning_velocity from prediction resolution rate. CREATE GET /api/oem/personality endpoint. MODIFY `static/js/today.js` — show a one-line personality summary in the morning brief ("Your organization decides quickly, takes moderate risks, and learns steadily."). MODIFY `static/js/learn.js` — add a "Who your organization is" section.

API contract

GET /api/oem/personality → returns JSON: { "dimensions": { "decision_velocity": {"value": 0.72, "label": "fast", "evidence_count": 14, "basis": "avg approval delay 2.1 days across 14 gates"}, ... 6 dimensions ... }, "summary": "Your organization decides quickly, takes moderate risks, and learns steadily. Knowledge is moderately siloed. Review discipline is strong.", "confidence": 0.68, "last_updated": "..." }. Each dimension MUST have evidence_count > 0 and a human-readable basis string explaining how it was inferred.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/personality. Response must have 6 dimensions, each with value (0-1), label, evidence_count > 0, and basis. 2) Auditor verifies no dimension is hardcoded — each basis must reference real model data (signal counts, timing, etc.). 3) Auditor opens TODAY and verifies a one-line personality summary appears. 4) Auditor verifies the summary is non-generic (not "Your organization is balanced" — must reference at least 2 specific dimensions). 5) Auditor verifies no survey/form UI exists anywhere (grep for "survey", "questionnaire" in static/).

Score delta

+1.0 (9.0 → 10.0 capped at 9.5). This is the V3 "organizational self-awareness" feature. Without it, Maestro cannot feel like it "understands" the company.

Estimated effort

2 days (1 day inference engine + 0.5 day API + 0.5 day frontend)

Feature #3 — Time-Axis Insight Layer (Past / Present / Future)

FEATURE #3 TimeAxisEngine: every insight exists across past/present/future, not just now

V3 Law embodied

V3: "Make Time Visible. Every insight should exist across time. Past, Present, Future. Example: Documentation quality — Past: Improved rapidly. Present: Stable. Future: Likely to decline within six weeks because onboarding quality is falling."

Current codebase gap

The current model is point-in-time. `/api/oem/dashboard` returns current metrics. `/api/oem/learning` returns calibration history (past only). `/api/oem/simulator` returns future prediction (hire_count only). No endpoint synthesizes past + present + future for a single domain or insight. V3 demands time visibility — "everything should feel alive" means seeing trajectory, not just snapshot.

Files to create/modify

CREATE `backend/maestro_oem/time_axis.py` — the TimeAxisEngine class. For a given domain (e.g., "payments", "auth", "deployment"), synthesize: `past` (trend over last 90 days from signal history + learning objects), `present` (current state from model), `future` (prediction from prediction_lifecycle + simulation, with confidence + basis). CREATE GET /api/oem/time-axis?domain=payments endpoint. MODIFY `static/js/today.js` — the "one thing learned overnight" item shows past/present/future for its domain. MODIFY `static/js/drill_down_modal.js` — the "Timeline" tab now shows past/present/future narrative, not just event timestamps.

API contract

GET /api/oem/time-axis?domain=payments → returns JSON: { "domain": "payments", "past": { "trend": "improving", "detail": "P1 incidents dropped from 3 to 1 over 90 days. Merge velocity increased 40%.", "data_points": 47 }, "present": { "state": "stable", "detail": "1 active P1, 2 open PRs, no bottlenecks detected.", "confidence": 0.82 }, "future": { "prediction": "likely stable for 4-6 weeks, then slight risk if the auth-service refactor delays the token refresh.", "confidence": 0.61, "basis": "linked law L-0001 (3 validations), dependency on auth domain", "time_horizon": "4-6 weeks" } }. Must work for any domain that has > 5 signals.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/time-axis?domain=payments. Response must have past, present, future with non-empty detail strings. 2) Auditor verifies past.data_points > 0 (real history, not hardcoded). 3) Auditor verifies future.prediction is a sentence (not a metric). 4) Auditor calls with a domain that has < 5 signals (e.g., "nonexistent") — must return 404 or a clear "insufficient data" response. 5) Auditor opens TODAY and verifies the "one thing learned" item references trajectory (past/future), not just present state.

Score delta

+0.5 (9.5 → 10.0 capped). V3's "make time visible" is the difference between a dashboard and a living system.

Estimated effort

1.5 days (1 day engine + 0.5 day API + frontend)

Feature #4 — Conversational Ask (LLM-Backed, Not Keyword Search)

FEATURE #4 ConversationalAsk: replace keyword search with LLM-synthesized answers from OEM evidence

V3 Law embodied

V3: "Replace Search with Conversation. Eventually every page becomes conversational." Law 9: "Every feature must reduce thinking while increasing judgment."

Current codebase gap

The current `/api/oem/ask` endpoint (decision.py answer_question) is keyword substring search — splits the query, matches tokens > 3 chars against law statements. This was honestly documented in round 5 but never upgraded. V3 explicitly demands conversation: "Why are releases slowing?" → "There are three causes. Would you like the executive explanation or the engineering explanation?" The current system cannot produce this. It returns a list of keyword-matched laws.

Files to create/modify

CREATE `backend/maestro_oem/conversational_ask.py` — the ConversationalAskEngine. Uses the existing `maestro_llm/` provider router (openai/anthropic/ollama) to synthesize answers. Pipeline: (1) retrieve relevant evidence via the existing autocomplete engine (semantic-ish retrieval), (2) compose a context window with the evidence + linked laws + "so what" (Feature #1), (3) call the LLM with a system prompt that enforces V3's voice ("We've seen this pattern before. Here's what usually happens. Here's why. Here's what I'd prepare. Approve?"), (4) return the synthesized answer + citations + confidence. MODIFY `backend/maestro_api/routes/oem.py` — add GET /api/oem/ask/conversational?q=... (keep the old /ask for backward compat). MODIFY `static/js/ask_v2.js` — use the conversational endpoint; show a multi-turn conversation UI (not a single Q&A;). MODIFY `backend/maestro_llm/providers.py` — ensure the LLM router is wired (it exists but may need a default provider).

API contract

GET /api/oem/ask/conversational?q=Why+are+releases+slowing → returns JSON: { "answer": "Releases are slowing for three reasons. First, the payments-edge team has 3 P1 incidents active, pulling focus from new work. Second, the auth-service OAuth consolidation (PR #102) has been in review for 6 days — longer than your usual 2-day review cycle. Third, sprint velocity dropped to 42 points last sprint, below your 6-sprint average of 58. Would you like me to prepare a recommendation for unblocking the auth review?", "citations": [{"type": "law", "code": "L-0001", "statement": "..."}, {"type": "signal", "artifact": "jira:EMEA-1247"}], "confidence": 0.74, "follow_up_questions": ["Prepare a recommendation for unblocking auth review?", "Show me the evidence for the velocity drop?"], "evidence_count": 8 }. If no LLM is configured, fall back to the existing keyword search with a clear "running in basic mode" flag (honest degradation, not silent).

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/ask/conversational?q=Why+are+releases+slowing. If an LLM is configured (check MAESTRO_LLM_PROVIDER env), response must have a multi-sentence synthesized answer (not a keyword-match list). 2) Response must have follow_up_questions (at least 1). 3) Response must have citations with real entity references. 4) If no LLM configured, response must have a "basic_mode": true flag and fall back to keyword search honestly. 5) Auditor opens ASK v2 surface and verifies the UI shows a conversation (multi-turn), not a single Q&A; card. 6) Auditor greps for "keyword" in ask_v2.js — must not appear in user-facing strings (the system is conversational, not search).

Score delta

+1.0 (9.5 → 10.0 capped). This is the V3 "replace search with conversation" feature. It is the single biggest user-experience shift — from querying a database to conversing with an intelligence.

Estimated effort

2 days (1 day LLM integration + 0.5 day conversation state + 0.5 day frontend). Requires an LLM API key in the environment (OpenAI/Anthropic/Ollama).

Feature #5 — Narrative Dashboard Replacer (Stories, Not Charts)

FEATURE #5 — **NarrativeReplacer: replace the 19 deep-surface charts with single-paragraph explanations**

V3 Law embodied

Law 1: "Software disappears. The interface should shrink as intelligence grows." V3: "Replace Dashboards with Narratives. Never display Knowledge Flow 73%. Instead: Engineering discovered something yesterday. Finance has not benefited yet. Three projects will likely repeat the same mistake."

Current codebase gap

The 19 deep surfaces (Home, Hayek, Knowledge Flow, etc.) still display charts, metrics, confidence bars, and law codes. The 4 meta-surfaces (TODAY, WORK, ASK, LEARN) are narrative, but they are the minority. A user who opens the command palette and navigates to "Knowledge Flow" sees duplicate-work cards with provider badges — not a narrative. V3 demands every dashboard become a story. The narrative.py engine exists but only powers the daily briefing, not the deep surfaces.

Files to create/modify

CREATE `backend/maestro_oem/narrative_replacer.py` — takes a surface name + the surface's data and returns a narrative paragraph + 1-3 key story items. For each of the 19 deep surfaces, define a narrative template that uses the "so what" engine (Feature #1). Example for Knowledge Flow: "Engineering discovered a pattern for retry logic yesterday. The payments team has not adopted it yet. Three upcoming PRs will likely repeat the same mistake. Here is the conversation that prevents it." CREATE GET /api/oem/narrative?surface=flow endpoint. MODIFY the 19 deep-surface JS files — replace chart/metric rendering with a call to the narrative endpoint + a "show details" expander (not a chart). Keep the raw data accessible via a "View raw data" link (for power users), but the default view is the narrative.

API contract

GET /api/oem/narrative?surface=flow → returns JSON: { "surface": "flow", "headline": "Engineering discovered a retry-logic pattern. Payments hasn't adopted it yet.", "narrative": "Yesterday, the platform team merged a retry-logic implementation in payments-edge (PR #448). The pattern is reusable across 3 upcoming PRs in the auth and platform domains. If not adopted, those PRs will likely reintroduce the same failure mode that caused the Nov 9 incident. The conversation that prevents it: link to Slack thread.", "story_items": [{"title": "Retry-logic pattern", "domain": "payments", "status": "discovered", "action": "share with auth team"}], "raw_data_available": true }. Must work for surface in {home, flow, hayek, physics, contradictions, predictions, assumptions, customer, ...}.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/narrative?surface=flow. Response must have headline (1 sentence), narrative (3-5 sentences), story_items (1-3 items). 2) Auditor verifies the narrative references real model data (not hardcoded). 3) Auditor opens the Knowledge Flow surface via command palette and verifies the DEFAULT view is a narrative paragraph, not a chart. 4) Auditor verifies a "View raw data" link exists that shows the old chart view. 5) Auditor calls with surface=nonexistent — must return 404. 6) Auditor greps the 19 deep-surface JS files for chart-rendering code (e.g., conf-bar, formatConfidence in user-facing divs) — at least 10 of 19 must have the chart as the SECONDARY view (narrative first).

Score delta

+0.5 (10.0 capped at 9.5). V3's "replace dashboards with narratives" applied to the deep surfaces. Combined with Feature #6 (humanize), this closes the round-10 gap.

Estimated effort

1 day (0.5 day engine + 0.5 day frontend rewrites for 10 surfaces; the other 9 can follow)

Feature #6 — Deep-Surface Humanize (Close the Round-10 Gap)

FEATURE #6 Apply humanize() to the 19 deep surfaces — strip law codes, confidence, receipts from visible text

V3 Law embodied

Law 7: "Never show machinery. Never expose OEM, Learning Objects, Receipts, Organizational Laws, Pattern IDs, Internal confidence, Prediction IDs, Receipt IDs, Evidence IDs."

Current codebase gap

The round-10 review found that `humanize()` was built (63 lines, comprehensive) but only called by 3 files (ask_v2.js, learn.js, today.js). The 19 deep surfaces still display raw internal vocabulary: `physics_laws.js` shows "L-0001" law codes and confidence percentages; `home_core.js` shows "Confidence: 38%" and "Linked laws: L-0001, L-0002"; `eng_audit.js` shows "receipts" directly. The utility exists but is not applied. This is the same "built but not applied" pattern from rounds 7 and 10.

Files to create/modify

MODIFY `static/js/physics_laws.js` — wrap visible law code display in `humanize()` (strip L-XXXX from visible text, keep it in data-law-code attribute for functionality). Replace `formatConfidence(l.confidence)` with confidence-as-story ("We've seen this 3 times" not "38%"). Rename button-label function calls from `contradictLaw` to `contradictPattern` in the UI (keep the function name internally). MODIFY `static/js/home_core.js` — wrap "Linked laws: L-0001, L-0002" in `humanize()` → "Based on 2 organizational patterns." Replace "Confidence: 38%" with the story form. MODIFY `static/js/eng_audit.js` — replace "receipts" with "signals" in all user-facing strings. MODIFY the remaining 16 deep-surface JS files — audit each for internal vocabulary in user-facing strings, wrap raw API text in `humanize()`.

API contract

No new API. This is a frontend-only fix. The acceptance is: no user-visible string in any of the 19 deep surfaces contains "L-XXXX", "confidence: X.XX", "receipt", "learning object", "evidence graph", "OEM" (as a user-facing label, not a code comment).

Acceptance test (auditor will run)

1) Auditor greps all 19 deep-surface JS files for user-facing strings (in template literals, innerHTML, textContent) containing: "L-\d{4}", "confidence:", "receipt", "learning object", "evidence graph". Count must be 0 (excluding comments and variable names). 2) Auditor opens each deep surface via command palette and visually verifies no internal vocabulary appears in the rendered UI. 3) Auditor verifies law codes still work as functional identifiers (drill-down still opens, contradict still works) — functionality preserved, vocabulary hidden. 4) Auditor runs the existing test suite — 0 regressions.

Score delta

+0.5 (closes the R10 gap, 8.5 → 9.0). This is the fix the coder should have done in round 10. It is the smallest feature but the most embarrassing gap.

Estimated effort

0.5 day (mechanical application of humanize() + confidence-as-story replacement across 19 files)

Feature #7 — Quarterly Evolution Report (Organization Becomes Wiser)

FEATURE #7 EvolutionReport: "How has our organization changed?" — the V3 end-state metric

V3 Law embodied

Law 10: "The organization should become progressively smarter. Not merely faster." V3: "Organizational Evolution. The product should eventually answer: How has our organization changed? Example: Your organization became 11% better at decision making." V3 final sentence: "The purpose of Maestro is not to help organizations do more work. The purpose of Maestro is to help organizations become wiser over time."

Current codebase gap

The codebase has no evolution report. The `/api/oem/learning` endpoint returns calibration history (Brier score, prediction accuracy) — but only for predictions, not for the organization as a whole. The `instrumentation.py` weekly snapshot collects metrics but does not synthesize them into an evolution narrative. V3 demands the product answer "How has our organization changed?" with specific deltas. This is the V3 end-state metric — the proof that Maestro makes organizations wiser, not just faster.

Files to create/modify

CREATE `backend/maestro_oem/evolution_report.py` — the EvolutionReportEngine. Compares the organization's state 90 days ago (from instrumentation snapshots + signal history) to now. Synthesizes 5 evolution dimensions, each with a delta + narrative: (1) decision_making (from approval bottleneck timing + decision velocity), (2) knowledge_discipline (from documentation + knowledge_death rates), (3) cross_functional_trust (from contradiction resolution + coordination signals), (4) knowledge_mobility (from cross-domain signal overlap), (5) prediction_accuracy (from Brier score trend). Each dimension: "improved"/"declined"/"stable" + percentage + narrative. CREATE GET /api/oem/evolution?window=90d endpoint. CREATE `static/js/evolution.js` — a new surface (accessible via command palette, NOT the sidebar — the sidebar stays at 5 items). MODIFY `static/js/learn.js` — add a "How you've evolved" section linking to the evolution surface. Depends on Feature #2 (Personality) for baseline comparison.

API contract

GET /api/oem/evolution?window=90d → returns JSON: { "window": "90d", "headline": "Your organization became 11% better at decision making. Documentation discipline increased. Cross-functional trust recovered.", "dimensions": [{ "name": "decision_making", "delta": "+11%", "direction": "improved", "narrative": "Average decision velocity dropped from 4.2 days to 3.1 days. The payments bottleneck cleared after the circuit-breaker PR.", "evidence_count": 47 }, ... 5 dimensions ...], "overall": "Your organization is becoming wiser, not just faster. Prediction accuracy improved 8%. Knowledge mobility accelerated. Meeting dependency declined.", "caveats": ["Only 90 days of data available; trends will solidify at 180 days."] }. Must have at least 3 dimensions with non-zero evidence_count.

Acceptance test (auditor will run)

1) Auditor calls GET /api/oem/evolution?window=90d. Response must have 5 dimensions, each with delta (signed percentage or "stable"), direction, narrative (2-3 sentences), evidence_count > 0. 2) Auditor verifies the narrative references real model data (not hardcoded). 3) Auditor verifies "overall" is a synthesized sentence (not a metric). 4) Auditor verifies "caveats" is non-empty (honest about data limitations). 5) Auditor opens the Evolution surface via command palette (Ctrl+K → "evolution") and verifies it renders the report. 6) Auditor verifies the sidebar still has 5 items (evolution is NOT in the sidebar — it is command-palette-only, per Law 1).

Score delta

+1.0 (9.5 → 10.0 capped). This is the V3 end-state metric. It is the proof that Maestro fulfills its purpose: "help organizations become wiser over time." Without it, the product is a dashboard with stories. With it, the product is a Living Intelligence Layer.

Estimated effort

1.5 days (1 day engine + 0.5 day API + frontend). Depends on Feature #2 (Personality) for baseline. Depends on instrumentation snapshots having 90 days of data (they do — the weekly snapshot loop has been running).

Build Order and Dependencies

Phase	Feature	Why This Order	Unlocks
1	#6 Deep-Surface Humanize	Closes R10 gap immediately. Smallest effort. Builds confidence.	Clean 9.0
2	#1 So What? Engine	Foundational. Features #3, #4, #5 all call it. Build first.	9.0 → 9.5
3	#2 Organizational Personality	No dependencies. Needed for #7 (evolution baseline).	9.5 → 10.0 cap
4	#3 Time-Axis Insight	Depends on #1 (so what). Adds past/present/future.	10.0 cap
5	#4 Conversational Ask	Depends on #1 (so what) + #2 (personality context). Biggest UX shift.	10.0 cap
6	#5 Narrative Replacer	Depends on #1 (so what) + #6 (humanize). Replaces deep-surface charts.	10.0 cap
7	#7 Evolution Report	Depends on #2 (personality baseline) + 90 days of snapshots. Time-based.	10.0 cap

Phase 1 (#6) is half a day and closes the round-10 gap. Do it first — it is the smallest, most verifiable, and most embarrassing gap. Once it is done, the score is 9.0 and every subsequent feature adds to a clean baseline.

Phase 2 (#1) is the foundation. The SoWhatEngine is called by Features #3, #4, and #5. Build it second. Once it exists, the other features compose it rather than duplicating logic.

Phases 3-7 can be parallelized after #1 and #2 are done, but the coder should build them sequentially for verification clarity. Each feature has an acceptance test — run it before claiming the feature is delivered. The recurring pattern across 10 rounds is "built but not applied" — these features must be APPLIED (the acceptance tests check application, not existence).

The Recurring Pattern to Break (Read This Twice)

BUILT ≠ APPLIED

Across 10 rounds, the coder has exhibited a consistent pattern: build the tool, claim it is delivered, do not apply it. Examples:

• Round 7: Added 4 meta-surfaces on top of 22 old ones. Claimed "22 → 4." Actually 23. Built the surfaces, did not collapse the old ones.

• Round 8: Built `escapeJs()`. Applied it to most onclick handlers. Missed 3 (p.provider, job.job_id, s.type). Fixed in round 5 after auditor flagged it.

• Round 10: Built `humanize()` utility (63 lines, comprehensive). Applied it to 1 file (ask_v2.js). Claimed "available to all surfaces." 19 deep surfaces still expose raw vocabulary. The utility existed; it was not called.

• Round 10: Built tenant isolation guard. It is a no-op in single-tenant mode (the default). The guard exists; it does not enforce.

For V3 features, the acceptance tests are designed to catch this pattern. Every test checks that the feature is APPLIED — not just that the file exists. Feature #6 requires grep to find 0 internal vocabulary in user-facing strings. Feature #1 requires the so_what field to be populated on actual recommendations. Feature #2 requires evidence_count > 0 on every personality dimension. Feature #4 requires a multi-sentence synthesized answer (not a keyword list).

The coder's checklist before claiming any V3 feature is delivered:

1. Did I run the acceptance test myself? (Not "will it pass" — did I RUN it?)

2. Did I check APPLICATION, not just EXISTENCE? (The file exists AND it is called/wired/rendered.)

3. Did I run the FULL test suite (not a subset)? (CI will catch this now, but run it locally first.)

4. Did I verify with a LIVE API call, not just code inspection? (The endpoint returns real data, not a hardcoded stub.)

5. Did I check the user-facing UI, not just the backend? (The feature is visible to a user, not just callable via API.)

If any answer is "no," the feature is not delivered. Do not claim it. The auditor will find the gap. The CI pipeline will find the gap. Save everyone time: apply, do not just build.

V3 Law Coverage Matrix

Constitution V3 has 10 Laws. The 7 features cover 6 of them directly. The remaining 4 Laws are either already satisfied (Law 1 sidebar collapse) or are ongoing principles (Law 5 internal vs external complexity) that apply to every feature.

V3 Law	Status	Feature(s) that satisfy it
Law 1: Software disappears (interface shrinks)	**PARTIALLY SATISFIED**	Sidebar collapsed (R8). Feature #5 (narrative replacer) completes it for deep surfaces.
Law 2: Organization feels alive	**GAP**	Feature #2 (Personality) + Feature #7 (Evolution) — org has identity + trajectory.
Law 3: Everything has memory	**SATISFIED**	Existing: prediction_lifecycle.py, learning.py, receipt.py, evidence_graph.py. All insights are
Law 4: Memory evolves into judgment	**GAP**	Feature #1 (So What?) — memory becomes actionable judgment via consequence synthesis.
Law 5: Internal complexity ↑ reduces external complexity ↓	**ONGOING PRINCIPLE**	Every feature must reduce UI. Acceptance tests verify (e.g., Feature #5 replaces charts with pa
Law 6: Organizations evolve; Maestro evolves with them	**GAP**	Feature #2 (Personality) + Feature #7 (Evolution Report).
Law 7: Never show machinery	**PARTIALLY SATISFIED**	Feature #6 (Deep-Surface Humanize) completes it. humanize() utility exists from R10.
Law 8: Everything answers "so what?"	**GAP**	Feature #1 (So What? Engine) — the foundational V3 feature.
Law 9: Reduce thinking, increase judgment	**GAP**	Feature #4 (Conversational Ask) — replaces search (thinking) with synthesis (judgment).
Law 10: Organization becomes progressively smarter	**GAP**	Feature #7 (Evolution Report) — the proof.

After all 7 features: 6 of 10 Laws fully satisfied, 2 partially satisfied (completing), 2 ongoing principles. Constitution adherence reaches 9.5/10. The remaining 0.5 (ambient integration via Chrome extension) is post-pilot.

Final Charge to the Coder

Constitution V3 is not a wishlist. It is a build instruction. The 7 features above are the engineering work that turns Maestro from "a dashboard with calm CSS" into "a Living Intelligence Layer." Each feature is grounded in the actual codebase — I read the 49 backend modules and 19 frontend files before specifying them. Nothing here is vapour.

The acceptance tests are not suggestions. They are the definition of done. A feature is not delivered when the file is created. It is delivered when the acceptance test passes AND the test is run by the coder (not just by the auditor). The recurring pattern across 10 rounds — build the tool, claim it is delivered, do not apply it — ends here. CI will catch subset-test-reporting. The acceptance tests will catch built-but-not-applied. The auditor will catch everything else.

Build order: #6 → #1 → #2 → #3 → #4 → #5 → #7. ~9.5 days of work. When all 7 are verified, Constitution adherence is 9.5/10 and the product genuinely is a Living Intelligence Layer — not a dashboard, not a copilot, not enterprise software. An organization that has become self-aware.

The V3 litmus test for every commit: "Does this make the organization wiser over time?" If yes, ship it. If no, redesign it. If it only makes the organization faster, reject it. V3 is not about speed. V3 is about wisdom. Build that.